



SVEUČILIŠTE JOSIPA JURJA STROSSMAYERA U OSIJEKU

FAKULTET PRIMIJENJENE MATEMATIKE I INFORMATIKE

Sveučilišni diplomski studij matematike
modul: Matematika i računarstvo

Primjena temporalne baze podataka u sustavu za praćenje prtljage u zračnom prometu

DIPLOMSKI RAD

Mentor:

doc. dr. sc. Mateja Đumić

Student:

Ana Koturić

Osijek, 2025.

Sadržaj

1	Uvod	1
2	Baze podataka	2
2.1	Tipovi baza podataka	2
2.2	Relacijske baze podataka	4
3	Temporalne baze podataka	6
3.1	Definicija i osnovni koncepti	6
3.2	Vremenske dimenzije	7
3.3	Modeliranje temporalnih podataka u relacijskom modelu	7
3.4	Podrška u sustavu PostgreSQL	8
3.5	Vrste temporalnih upita	8
4	Primjena temporalne baze podataka	9
4.1	Dizajn baze i MEV dijagram	9
4.1.1	Kreiranje tipova	10
4.1.2	Kreiranje tablica	11
4.1.3	Kreiranje indeksa	13
4.2	Implementacija temporalne logike	14
4.2.1	Kreiranje okidača	15
4.2.2	Kreiranje funkcija	16
4.3	Upiti	18
	Literatura	29
	Sažetak	30
	Summary	31
	Životopis	32

1 | Uvod

Iako se baze podataka koriste već desetljećima, potreba za njima nikad nije bila veća. U današnjem svijetu, u kojem se podaci stvaraju ogromnom brzinom, veliki broj usluga nam je omogućen upravo zbog baza podataka. One su temelj aplikacija, web-stranica, transakcija i raznih sustava, a omogućuju i analize koje vode boljim odlukama.

Unatoč svim svojim prednostima, većina baza je ograničena na pohranu samo trenutnog stanja podataka. To znači da se prilikom svake izmjene staro stanje nepovratno gubi, što često predstavlja veliki problem u modernim sustavima gdje je povijest događaja od presudne važnosti. Kao odgovor na ovaj problem, razvijen je koncept temporalnih baza podataka koje su dizajnirane s ugrađenom podrškom za upravljanje podacima u vremenskoj dimenziji.

Sustav za praćenje prtljage u zračnom prometu, savršen je primjer sustava u kojem se stanje podataka neprestano mijenja kroz vrijeme i prostor te je povijest događaja od presudne važnosti. Svaka promjena statusa prtljage (kao prijava, ukrcaj ili preuzimanje) bilježi se kao novi događaj, što zahtijeva očuvanje cjelokupne povijesti prtljage. Upravo je ta potreba za praćenjem i analizom povijesti kretanja prtljage ključan razlog za odabir ovog primjera i idealna demonstracija korisnosti temporalnih baza podataka u odnosu na standardne baze podataka.

Ovaj diplomski rad bavi se temporalnim bazama podataka i demonstrira njihovu korisnost. Drugo poglavlje opisuje baze podataka općenito, predstavlja glavne tipove baza podataka i njihove karakteristike s posebnim naglaskom na relacijske baze. Treće poglavlje detaljno objašnjava temporalne baze podataka, njihove osnovne koncepte, vremenske dimenzije i modeliranje temporalnih baza. Četvrto poglavlje fokusirano je na praktičnu primjenu i implementaciju temporalnog modela za praćenje prtljage u zračnom prometu.

2 | Baze podataka

Baza podataka je zajednička zbirka logički povezanih podataka, dizajnirana da zadovolji informacijske potrebe jedne organizacije [1]. Dobro dizajnirana i učinkovita baza podataka ključna je za rad svake organizacije jer su one u pozadini web-stranica, aplikacija, transakcija i internih procesa. Baze podataka pohranjuju zapise, izvršavaju operacije, prate stanja i povijest, te omogućuju analize koje vode boljim odlukama i unapređenju poslovanja.

Za upravljanje bazom podataka zadužen je sustav za upravljanje bazom podataka (SUBP; engl. *Database Management System* - DBMS). SUBP je kompleksan programski sustav koji služi kao sučelje između korisnika ili aplikacije i same baze podataka. On pruža sve potrebne alate za definiranje strukture baze, unos, ažuriranje, brisanje i dohvaćanje podataka.

Osim tih osnovnih operacija, SUBP je odgovoran i za ključne, napredne funkcije. To uključuje nadzor pristupa koji kontrolira tko može pristupiti podacima i koje operacije smije izvršiti, zatim osiguranje integriteta kojim se jamči da su podaci u bazi točni i dosljedni, sukladno definiranim pravilima. Također, jedna od naprednih funkcija SUBP-a je oporavak od grešaka koji omogućuje vraćanje baze u ispravno stanje u slučaju hardverskih ili softverskih kvarova. SUBP je zadužen i za upravljanje istovremenim pristupom čime rješava probleme koji mogu nastati kada više korisnika istovremeno pokušava mijenjati iste podatke.

2.1 Tipovi baza podataka

Različite baze podataka pohranjuju podatke na različite načine, pa tako prema načinu pohrane razlikujemo dva glavna tipa baza podataka [2]. Prvi i najrašireniji tip su relacijske baze podataka u kojima su podaci organizirani u tablice sa strogo definiranom strukturom. Zbog svoje pouzdanosti i osiguravanja integriteta podataka, predstavljaju standard za većinu transakcijskih sustava.

Drugi tip baza podataka su NoSQL (engl. *Not Only SQL*) baze podataka koje su dizajnirane za upravljanje velikim količinama podataka. Njihova snaga leži u rukovanju različitim tipovima podataka (strukturiranim, polustrukturiranim i nestrukturiranim) u fleksibilnim formatima, često bez unaprijed definirane strukture. Ključna prednost im je podrška za horizontalno skaliranje (dodavanje više

poslužitelja), što ih čini idealnim za *big data*¹ analitiku, IoT² aplikacije i sustave za upravljanje sadržajem. Najpoznatiji tipovi NoSQL baza su:

- Ključ-vrijednost baze podataka - pohranjuju podatke u jednostavnim parovima ključa i vrijednosti.
- Grafovske baze podataka - optimizirane za pohranu i pretraživanje podataka s kompleksnim međusobnim vezama.
- Stupčane baze podataka - pohranjuju podatke po stupcima umjesto po redcima, što ih čini idealnim za analitičke upite.
- Dokument baze podataka - pohranjuju podatke u fleksibilnim dokumentima, najčešće u JSON formatu.

Oba tipa, relacijske baze podataka i NoSQL baze podataka, imaju svoje prednosti i nedostatke, te su namijenjene za rješavanje različitih vrsta problema. Njihove ključne razlike sažete su u Tablici 2.1 [7].

Uz relacijske i NoSQL baze podataka, u praksi se susreću i posebni tipovi baza podataka kao što su:

- Temporalne baze podataka - sadrže podršku za praćenje i dohvaćanje podataka kroz vrijeme. Dok standardne baze pohranjuju samo trenutno stanje, temporalne baze čuvaju i cjelokupnu povijest promjena.
- Prostorne baze podataka - optimizirane su za pohranu, indeksiranje i postavljanje upita nad geografskim podacima (npr. karte, rute, lokacije).
- Time-Series baze podataka - specijalizirane su za podatke koji se mjere i prikupljaju u pravilnim vremenskim intervalima, poput očitavanja sa senzora, telemetrijskih podataka ili kretanja cijena na financijskim tržištima.

Temporalne i prostorne baze podataka često dolaze kao proširenje relacijskih baza podataka.

¹Big data je izraz koji se odnosi na iznimno velike i složene skupove podataka.

²IoT (engl. Internet of Things) je koncept povezivanja svakodnevnih fizičkih objekata na internet.

	Relacijska (SQL) baza podataka	NoSQL baza podataka
Model podataka	Tablični format s recima i stupcima	Ključ-vrijednost, dokument, grafovski, stupčani model
Skalabilnost	Vertikalno skalabilna (nadogradnja jednog poslužitelja)	Horizontalno skalabilna (dodavanje više poslužitelja)
Jezik upita	SQL (standardiziran)	Jezici se razlikuju među različitim bazama
ACID svojstva ¹	U potpunosti podržava ACID, osiguravajući pouzdanost i konzistentnost	Često ublažava ACID svojstva radi boljih performansi i dostupnosti
Količina podataka	Može imati poteškoća s rukovanjem ogromnim količinama podataka	Izvrсна u upravljanju masivnim skupovima podataka
Tip podataka	Radi isključivo sa strukturiranim podacima	Podržava polustrukturirane i nestrukturirane podatke
Najbolja primjena	Transakcijski sustavi (e-trgovina, bankarstvo) koji zahtijevaju strogi integritet podataka	Analitika u stvarnom vremenu, društvene mreže, IoT sustavi i aplikacije koje zahtijevaju fleksibilnost i skalabilnost

Tablica 2.1: Usporedba relacijskih i NoSQL baza podataka.

2.2 Relacijske baze podataka

Relacijske baze podataka, temeljene na modelu kojeg je 1970. godine predložio Edgar F. Codd, predstavljaju najrašireniji i najutjecajniji pristup pohrani strukturiranih podataka. Njihova popularnost proizlazi iz intuitivnosti i jednostavnosti samog modela: podaci se organiziraju u tablice, koje se sastoje od redaka (n-torki) i stupaca (atributa). Svaka tablica predstavlja jedan entitet iz stvarnog svijeta (npr. let). Svaki redak u toj tablici predstavlja instancu entiteta, tj. jedan konkretan primjerak tog entiteta (npr. jedan određeni let s brojem OU416). Kako bi se osiguralo da ne postoje dupli zapisi, svaki redak sadrži jedinstveni identifikator poznat kao primarni ključ, koji je zapravo atribut ili skupina atributa s jedinstvenom vrijednošću. Veze između tablica uspostavljaju se pomoću vanjskih ključeva (primarnih ključeva iz neke druge tablice), čime se

¹ ACID svojstva detaljnije su opisana u Poglavlju 2.2

osigurava integritet i dosljednost podataka u cijeloj bazi.

Za interakciju s podacima u relacijskim bazama koristi se standardizirani upitni jezik SQL (engl. *Structured Query Language*). Prije same fizičke implementacije, logička struktura i odnosi između entiteta vizualno se planiraju pomoću MEV dijagrama (model entitet-veza; engl. *Entity-Relationship* - ER), koji služi kao nacrt baze podataka. Osnovni elementi MEV dijagrama su entiteti (objekti o kojima čuvamo podatke, npr. let), njihovi atributi (svojstva, npr. broj leta) i veze koje opisuju kako su entiteti međusobno povezani.

Osim strogo definirane strukture, pouzdanost i integritet podataka u relacijskim bazama osigurani su kroz transakcije koje moraju zadovoljiti ACID svojstva. Transakcija predstavlja niz operacija koje se izvršavaju kao jedna logička cjelina. ACID je akronim za atomarnost (engl. *atomicity*), konzistentnost (engl. *consistency*), izolaciju (engl. *isolation*) i trajnost (engl. *durability*) [3].

Atomarnost jamči da će se transakcija izvršiti u potpunosti ili se neće izvršiti uopće. Ako bilo koji dio transakcije ne uspije, cijela transakcija se poništava i baza ostaje u nepromijenjenom stanju. Konzistentnost osigurava da svaka uspješna transakcija prebacuje bazu iz jednog ispravnog (konzistentnog) stanja u drugo. Sva definirana pravila i ograničenja baze moraju biti zadovoljena. Izolacija osigurava da se transakcije koje se izvršavaju istovremeno ne ometaju međusobno. Svaka transakcija se izvršava kao da je jedina u sustavu, tj. njezino izvršavanje ne utječe na rezultat druge transakcije koja se izvršava istovremeno. Trajnost jamči da su promjene napravljene uspješnom transakcijom trajno pohranjene i da će opstati čak i u slučaju kvara sustava (npr. nestanak struje).

3 | Temporalne baze podataka

Prethodno poglavlje opisalo je standardne (netemporalne) baze podataka, relacijske i NoSQL baze podataka, te njihove osnovne karakteristike i primjene. Iako svaka od tih baza ima svoje prednosti, ključno ograničenje standardnih baza je da su usmjerene na pohranu samo trenutnog stanja podataka. U takvim sustavima, kada se podatak obriše ili ažurira, staro stanje se nepovratno gubi. Upravo taj nedostatak rješavaju temporalne baze podataka, koje su dizajnirane s ugrađenom podrškom za upravljanje podacima u vremenskoj dimenziji.

3.1 Definicija i osnovni koncepti

Temporalna baza podataka je baza podataka koja pohranjuje podatke prateći njihove promjene kroz vrijeme. Dok standardne baze podataka prilikom svake izmjene nepovratno gube staro stanje, temporalne baze čuvaju i staro i novo stanje, pridružujući svakom od njih odgovarajući vremenski period valjanosti.

Glavna prednost ovakvog pristupa je mogućnost „putovanja kroz vrijeme” nad podacima. To omogućuje ne samo dohvaćanje trenutnog stanja, već i potpunu rekonstrukciju povijesti događaja, analizu trendova i postavljanje kompleksnih upita o prošlim stanjima, što je u standardnim bazama izuzetno teško ili nemoguće izvesti.

Implementacija temporalnih baza, unatoč svojim prednostima, donosi i određene izazove. Prvi i najočitiji je povećana potrošnja resursa za pohranu jer čuvanje cjelokupne povijesti podataka, a ne samo trenutnog stanja, dovodi do veće potrošnje prostora za pohranu. Svaka promjena generira novi zapis, što s vremenom može značajno povećati veličinu baze. Na taj se problem nadovezuje i problem složenosti dizajna sheme. Dizajniranje sheme za temporalnu bazu zahtijeva pažljivo planiranje kako bi se ispravno modelirala vremenska dimenzija. To uključuje definiranje intervala valjanosti i osiguravanje integriteta podataka kroz vrijeme, što povećava kompleksnost u odnosu na klasične modele. Kompleksniji modeli, često podrazumijevaju i složenije upite što vodi do problema zahtjevnije obrade upita. Umjesto jednostavnog dohvaćanja trenutnog stanja, oni često uključuju operacije s vremenskim intervalima, što može rezultirati zahtjevnijom obradom i potencijalno sporijim vremenom odziva.

3.2 Vremenske dimenzije

U teoriji temporalnih baza podataka razlikujemo tri osnovne vremenske dimenzije:

- Vrijeme valjanosti (engl. *valid time*) - predstavlja period u kojem je neka činjenica bila istinita u stvarnom svijetu. Na primjer, ako je prtljaga fizički stigla na lokaciju za sortiranje 25.8.2025. u 10:00 i napustila je 25.8.2025. u 10:15, vrijeme valjanosti za stanje „u sortirnici” je interval [2025-08-25 10:00, 2025-08-25 10:15). Praktični dio ovog rada bavi se isključivo modeliranjem vremena valjanosti.
- Vrijeme transakcije (engl. *transaction time*) - predstavlja period u kojem je neki podatak bio pohranjen u bazi podataka. Ono bilježi kada je informacija unesena, izmijenjena ili obrisana iz baze. Na primjer, podatak da je prtljaga bila u sortirnici mogao je biti unesen u bazu tek u 10:02 zbog kašnjenja sustava.
- Vrijeme odluke (engl. *decision time*) - predstavlja trenutak u kojem je donesena odluka o nekoj činjenici, a koji se može razlikovati i od vremena kada je ta činjenica postala važeća i od vremena kada je zabilježena u bazi. Na primjer, odluka o preusmjeravanju prtljage na drugi let donesena je u 11:00, ali je ta promjena u bazu unesena tek u 11:05.

Ovisno o tome koliko vremenskih dimenzija podržavaju, razlikujemo sljedeće temporalne tablice:

- Uni-temporalne - imaju jednu vremensku dimenziju; vrijeme valjanosti ili vrijeme transakcije.
- Bi-temporalne - imaju dvije vremenske dimenzije; vrijeme valjanosti i vrijeme transakcije.
- Tri-temporalne - imaju tri vremenske dimenzije; vrijeme valjanosti, vrijeme transakcije i vrijeme odluke.

3.3 Modeliranje temporalnih podataka u relacijskom modelu

Iako postoje specijalizirani sustavi, temporalna funkcionalnost se najčešće implementira unutar relacijskog modela. Najčešći pristup modeliranju vremena valjanosti je dodavanje dvaju stupaca u tablicu koja prati povijest: jednog koji označava početak perioda valjanosti (npr. *vrijedi_od*) i drugog koji označava kraj perioda valjanosti (npr. *vrijedi_do*).

Kada se dogodi novi događaj, prethodni, do tada aktivni redak se zatvara tako da se u njegov stupac *vrijedi_do* upiše vrijeme novog događaja. Nakon toga se unosi novi redak s novim stanjem, čiji period započinje tim istim vremenom. Kako bi se označilo da je novi redak trenutno aktivan, njegov stupac *vrijedi_do* se ostavlja prazan (*NULL*) ili se postavlja na neki datum u dalekoj budućnosti.

3.4 Podrška u sustavu PostgreSQL

PostgreSQL je izuzetno moćan sustav za upravljanje relacijskim bazama podataka koji nudi napredne mogućnosti idealne za implementaciju temporalne logike. U praktičnom dijelu ovog rada korištene su dvije njegove ključne funkcionalnosti koje ga čine prikladnim za problem temporalnosti.

Prva od njih je podrška za nativni tip podatka *TSTZRANGE*, koji je dizajniran za rad s vremenskim rasponima. Korištenje tipa *TSTZRANGE* (vremenski raspon s vremenskom zonom), umjesto uobičajenog pristupa s dva odvojena stupca za početak i kraj perioda, značajno pojednostavljuje upite i omogućuje korištenje specijaliziranih operatora za rad s intervalima (npr. provjera preklapanja, sadržavanja, spajanja).

Druga ključna funkcionalnost su GIST (Generalized Search Tree) indeksi i ograničenje isključenja (*EXCLUDE USING GIST*). Ograničenje isključenja osigurava da se u tablici ne mogu pojaviti dva retka koja zadovoljavaju određeni uvjet, poput preklapanja vremenskih intervala za isti komad prtljage. Time se na razini same baze podataka garantira integritet i točnost temporalnih podataka, sprječavajući unos nelogičnih i preklapajućih stanja.

3.5 Vrste temporalnih upita

Mogućnost postavljanja upita nad povijesnim podacima je temeljna prednost temporalnih baza. Neke od osnovnih vrsta upita koje ovakav model omogućuje, a koji su demonstrirani i u praktičnom dijelu rada, su:

- Upiti o trenutnom stanju - odgovaraju na pitanje „Gdje je prtljaga X sada?“. Ovi upiti jednostavno filtriraju retke kojima je gornja granica vremenskog intervala otvorena (npr. *upper(Validity_Period) IS NULL*).
- Upiti u točki u vremenu (engl. *point-in-time*) - odgovaraju na pitanje „Gdje je prtljaga X bila jučer u 15:00?“. Ovi upiti koriste operator sadržavanja (*@>*) kako bi pronašli interval koji je bio važeći u točno zadanom trenutku. Korištenje operatora sadržavanja (*@>*) ne bi bilo moguće bez *range* tipa podataka kao što je *TSTZRANGE*. U slučaju korištenja dva odvojena stupca, ekvivalentan upit bio bi znatno složeniji i manje čitljiv.
- Upiti nad periodom / rekonstrukcija povijesti - odgovaraju na pitanje „Prikaži cijelu povijest kretanja prtljage X“. Ovi upiti dohvaćaju sve povijesne retke za određeni entitet i redaju ih kronološki, čime se dobiva potpuna slika svih promjena stanja.

4 | Primjena temporalne baze podataka

Kao praktični dio rada, dizajniran je i implementiran model temporalne baze podataka za sustav praćenja prtljage u zračnom prometu. Ovaj primjer je odabran jer je stanje prtljage po prirodi promjenjivo – ono se neprestano mijenja kroz vrijeme i prostor, što ga čini idealnim za demonstraciju temporalnih koncepata. Kompletan kod za implementaciju baze je dostupan u GitHub repozitoriju na sljedećem linku <https://github.com/akoturic/Temporal-database->.

Standardni (netemporalni) pristup, koji pamti samo trenutnu lokaciju i status prtljage, nije dovoljan za rješavanje ključnih problema poput rekonstrukcije putanje, analize kašnjenja ili rješavanja slučajeva izgubljene prtljage. Upravo je potreba za čuvanjem i analizom cjelokupne povijesti kretanja ono što temporalne baze čini idealnim rješenjem.

Za tehničku implementaciju odabran je sustav za upravljanje bazama podataka *PostgreSQL*. Ovaj sustav odabran je zbog svoje napredne podrške za kompleksne tipove podataka, specifično za tip *TSTZRANGE*, te mogućnosti kreiranja *GIST* indeksa koji su optimizirani za temporalne upite, što je detaljnije objašnjeno u prethodnom poglavlju.

4.1 Dizajn baze i MEV dijagram

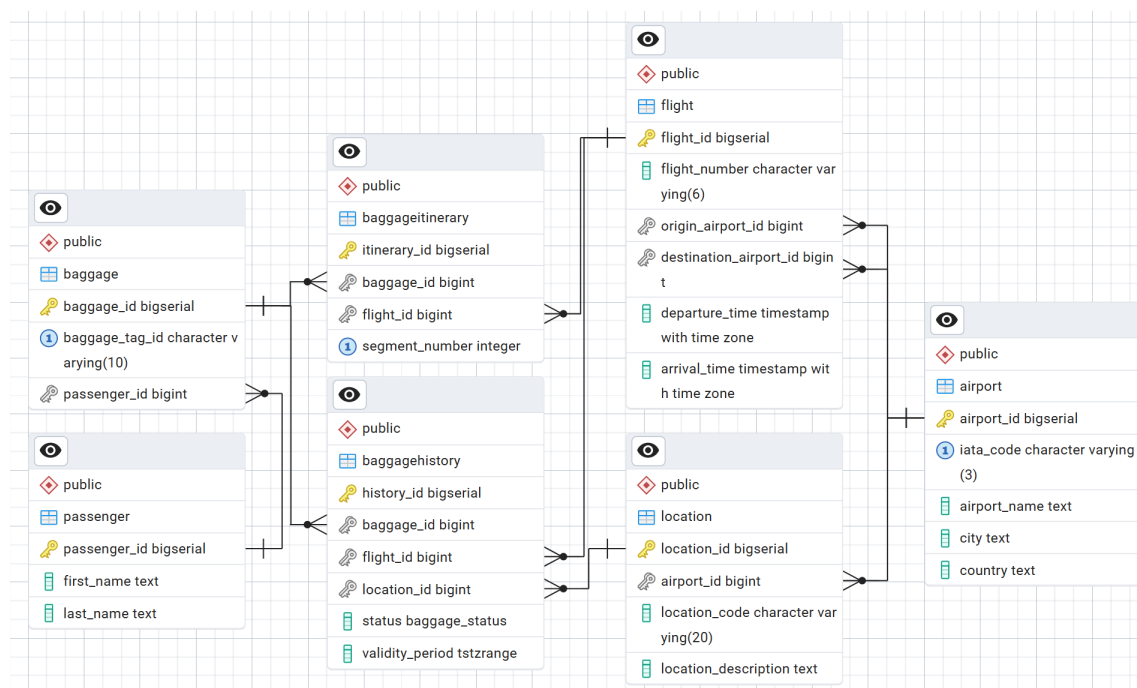
Temelj svakog kvalitetnog sustava je pažljivo dizajniran model podataka. Prije same implementacije, korištenjem MEV dijagrama, definirani su ključni entiteti, njihovi atributi i međusobne veze.

Glavni entiteti sustava su:

- Putnik (engl. *Passenger*) - sadrži osnovne podatke o putniku.
- Zračna luka (engl. *Airport*) - sadrži podatke o zračnim lukama.
- Let (engl. *Flight*) - definira let između dvije zračne luke s vremenima polaska i dolaska.
- Lokacija (engl. *Location*) - predstavlja specifičnu točku unutar zračne luke (npr. šalter za prijavu, sortirnica, izlaz za ukrcaj) ili izvan nje (npr. avion).

- Prtljaga (engl. *Baggage*) - predstavlja jedinstveni komad prtljage povezan s određenim putnikom.
- Plan puta prtljage (engl. *BaggageItinerary*) - definira planiranu rutu za komad prtljage, sastoji se od jednog ili više segmenata (letova).
- Povijest prtljage (engl. *BaggageHistory*) - ključna temporalna tablica koja čuva cjelokupnu povijest promjena statusa i lokacije za svaki komad prtljage.

Na temelju ovih entiteta, dizajniran je MEV dijagram koji služi kao nacrt za fizičku implementaciju baze (Slika 4.1).



Slika 4.1: MEV dijagram sustava za praćenje prtljage u zračnom prometu.

4.1.1 Kreiranje tipova

Prije kreiranja samih tablica, za potrebe modela definiran je i novi, korisnički definirani tip podatka `baggage_status` (Programski kod 4.1). Korištenjem `ENUM` tipa umjesto, primjerice, `VARCHAR` tipa za pohranu statusa prtljage, postiže se nekoliko bitnih prednosti.

Baza podataka dozvoljava unos isključivo predefiniranih vrijednosti, čime se spriječavaju greške pri unosu (npr. tipfeleri) i samim time osigurava integritet podataka. Kod i upiti su jasniji jer se koriste semantički nazivi umjesto brojeva ili stringova što poboljšava čitljivost i održivost. `ENUM` tipovi se interno pohranjuju kao integeri, što ih čini efikasnijima za pohranu i pretraživanje od stringova i na kraju poboljšava performanse.

```

1 CREATE TYPE baggage_status AS ENUM (
2     'CHECKED_IN',
3     'IN_SORTING_FACILITY',
4     'LOADED',
5     'IN_TRANSIT',
6     'UNLOADED',
7     'READY_FOR_PICKUP',
8     'CLAIMED',
9     'DELAYED',
10    'LOST'
11 );

```

Programski kod 4.1: SQL kod za kreiranje *ENUM* tipa *baggage_status*.

4.1.2 Kreiranje tablica

Na temelju MEV dijagrama, kreirane su odgovarajuće tablice koje čine bazu podataka. Mogu se podijeliti u dvije glavne skupine: standardne (netemporalne) tablice koje čuvaju osnovne entitete i ključnu temporalnu tablicu koja bilježi povijest.

Netemporalne tablice

Netemporalne tablice se u ovom modelu koriste za pohranu osnovnih podataka o putnicima, letovima, zračnim lukama i planiranom putu prtljage. One sadrže trenutno važeće informacije.

```

1 CREATE TABLE Passenger (
2     passenger_id BIGSERIAL PRIMARY KEY,
3     first_name TEXT NOT NULL CHECK (length(trim(first_name)) > 0),
4     last_name TEXT NOT NULL CHECK (length(trim(last_name)) > 0)
5 );
6
7 CREATE TABLE Airport (
8     airport_id BIGSERIAL PRIMARY KEY,
9     iata_code VARCHAR(3) UNIQUE NOT NULL CHECK (iata_code ~ '^[A-Z]{3}$'),
10    airport_name TEXT NOT NULL,
11    city TEXT NOT NULL,
12    country TEXT NOT NULL
13 );
14
15 CREATE TABLE Location (
16     location_id BIGSERIAL PRIMARY KEY,
17     airport_id BIGINT REFERENCES Airport(airport_id),
18     location_code VARCHAR(20) NOT NULL CHECK (length(trim(location_code)) > 0),
19     location_description TEXT
20 );
21
22 CREATE TABLE Flight (
23     flight_id BIGSERIAL PRIMARY KEY,

```

```

24     flight_number VARCHAR(6) NOT NULL CHECK (flight_number ~ '^[A-
25     origin_airport_id BIGINT NOT NULL REFERENCES Airport(airport_id
26     destination_airport_id BIGINT NOT NULL REFERENCES Airport(
27     departure_time TIMESTAMPTZ NOT NULL,
28     arrival_time TIMESTAMPTZ NOT NULL,
29     CONSTRAINT flight_times_check CHECK (departure_time <
30     CONSTRAINT flight_origin_destination_check CHECK (
31 );
32
33 CREATE TABLE Baggage (
34     baggage_id BIGSERIAL PRIMARY KEY,
35     baggage_tag_id VARCHAR(10) UNIQUE NOT NULL CHECK (
36     passenger_id BIGINT NOT NULL REFERENCES Passenger(passenger_id)
37 );
38
39 CREATE TABLE BaggageItinerary (
40     itinerary_id BIGSERIAL PRIMARY KEY,
41     baggage_id BIGINT NOT NULL REFERENCES Baggage(baggage_id),
42     flight_id BIGINT NOT NULL REFERENCES Flight(flight_id),
43     segment_number INT NOT NULL CHECK (segment_number > 0),
44     CONSTRAINT uq_baggage_segment UNIQUE (baggage_id,
45     CONSTRAINT uq_baggage_flight UNIQUE (baggage_id, flight_id)
46 );

```

Programski kod 4.2: SQL kod za kreiranje osnovnih tablica.

Kao što je vidljivo u Programskom kodu 4.2, osim definiranja osnovnih stupaca i tipova podataka, korištena su i brojna ograničenja (engl. *constraints*) kako bi se osigurala kvaliteta i integritet podataka. Primjerice, *CHECK* ograničenja koriste se za validaciju formata podataka (npr. *iata_code* mora sadržavati točno tri velika slova), dok *UNIQUE* ograničenja jamče jedinstvenost vrijednosti unutar stupca (npr. svaka oznaka prtljage *baggage_tag_id* mora biti jedinstvena). Vanjski ključevi osiguravaju da se u bazi ne mogu pojaviti zapisi koji referenciraju nepostojeće entitete (npr. let ne može imati nepostojeću polaznu zračnu luku).

Temporalne tablice

Tablica *BaggageHistory* srce je temporalnog modela. Njezina struktura prikazana je u Programskom kodu 4.3.

```

1 CREATE TABLE BaggageHistory (
2     history_id BIGSERIAL PRIMARY KEY,
3     baggage_id BIGINT NOT NULL REFERENCES Baggage(baggage_id),
4     location_id BIGINT NOT NULL REFERENCES Location(location_id),
5     flight_id BIGINT REFERENCES Flight(flight_id),
6     status baggage_status NOT NULL,

```

```
7     validity_period TSTZRANGE NOT NULL ,  
8     CONSTRAINT excl_overlapping_baggage_history  
9     EXCLUDE USING GIST (baggage_id WITH =, validity_period WITH &&)  
10 );
```

Programski kod 4.3: SQL kod za kreiranje temporalne tablice *BaggageHistory*.

Ključni element ove tablice je stupac *validity_period* tipa *TSTZRANGE*, koji čuva informaciju o vremenskom intervalu u kojem je određen status na određenoj lokaciji bio važeći. Kako bi se osiguralo da za jedan komad prtljage ne mogu postojati dva stanja koja se preklapaju u vremenu, korišteno je *GIST* ograničenje *EXCLUDE USING GIST*. Ovo ograničenje na razini baze jamči integritet temporalnih podataka.

4.1.3 Kreiranje indeksa

Kako bi se osiguralo brzo i efikasno izvršavanje upita, pogotovo nad velikim količinama podataka, baza podataka koristi indekse. Indeksi su posebne strukture podataka koje omogućuju brže pronalaženje redaka u tablici. U *PostgreSQL*-u, neki indeksi se kreiraju automatski prilikom definiranja određenih ograničenja, dok se drugi kreiraju ručno s ciljem optimizacije specifičnih upita.

Automatski kreirani indeksi

Prilikom definiranja nekih ograničenja, *PostgreSQL* automatski kreira odgovarajuće indekse kako bi osigurao njihovu efikasnu provedbu. U ovom modelu, to su *B-tree* indeksi i *GIST* indeksi.

B-tree indeksi kreiraju se automatski prilikom definiranja *PRIMARY KEY* i *UNIQUE* ograničenja i imaju dvostruku ulogu. Istovremeno osiguravaju jedinstvenost brzom provjerom da novi redak ne krši postavljeno pravilo i optimiziraju dohvaćanje podataka, omogućujući izuzetno brzo pronalaženje točno jednog retka na temelju vrijednosti primarnog ili jedinstvenog ključa (npr. dohvat zračne luke po *iata_code* -u).

EXCLUDE USING GIST ograničenje, korišteno na temporalnoj tablici *BaggageHistory*, također automatski kreira *GIST* indeks. Dok standardni *B-tree* indeksi nisu efikasni za pretraživanje vremenskih intervala, *GIST* indeksi su dizajnirani upravo za rad s kompleksnim tipovima podataka poput *TSTZRANGE*. Korištenje ovog indeksa ubrzava izvršavanje svih temporalnih upita, pogotovo *point-in-time* upita koji koriste operator sadržavanja (*@>*), i istovremeno osigurava integritet podataka provjeravajući da ne postoje preklapajući vremenski intervali za isti komad prtljage.

Ručno kreirani indeksi

Osim automatski kreiranih, za potrebe optimizacije specifičnih upita, ručno su kreirani i sljedeći indeksi koji su implementirani u Programskom kodu 4.4:

- Indeksi na vanjskim ključevima (npr. *BaggageItinerary(baggage_id)*) - iako ih *PostgreSQL* ne kreira automatski, dodavanje indeksa na stupce koji služe kao vanjski ključevi je standardna praksa koja ubrzava operacije spajanja tablica (*JOIN*).
- Indeksi za pretragu (npr. *Flight(flight_number)*) - kreirani su na stupcima za koje se očekuje da će se često koristiti u *WHERE* klauzuli upita.
- Jedinstveni parcijalni indeksi - ovo su napredni indeksi koji primjenjuju jedinstvenost samo na podskup redaka u tablici (one koji zadovoljavaju *WHERE* uvjet). Primjerice, *uq_bh_one_open* osigurava da za svaki komad prtljage može postojati samo jedan otvoreni vremenski interval (onaj gdje je *upper(ValidityPeriod)* *IS NULL*), što je ključno za integritet temporalne logike.

```

1  -- Indeksi na vanjskim ključevima za ubrzavanje JOIN operacija
2  CREATE INDEX ON BaggageItinerary (baggage_id);
3  CREATE INDEX ON BaggageItinerary (flight_id);
4
5  -- Indeksi za ubrzavanje pretrage po često korištenim atributima
6  CREATE INDEX ON Flight (flight_number);
7  CREATE INDEX ON BaggageHistory (status);
8  CREATE INDEX ON BaggageHistory (flight_id);
9
10 -- Jedinstveni parcijalni indeksi
11 CREATE UNIQUE INDEX uq_location_per_airport
12 ON Location (airport_id, location_code)
13 WHERE airport_id IS NOT NULL;
14
15 CREATE UNIQUE INDEX uq_location_global
16 ON Location (location_code)
17 WHERE airport_id IS NULL;
18
19 CREATE UNIQUE INDEX uq_bh_one_open
20 ON BaggageHistory(baggage_id)
21 WHERE upper(ValidityPeriod) IS NULL;
```

Programski kod 4.4: SQL kod za kreiranje indeksa.

4.2 Implementacija temporalne logike

U standardnim bazama, operacije se svode na jednostavne *INSERT*, *UPDATE* i *DELETE* naredbe. Međutim, u temporalnom modelu, gdje je cilj očuvanje povijesti, te operacije moraju biti pažljivo kontrolirane.

4.2.1 Kreiranje okidača

Kako bi se osigurala nepromjenjivost povijesnih podataka, na temporalnoj tablici *BaggageHistory* definirana su dva okidača (engl. *trigger*). U ovom modelu, njihova je uloga u potpunosti onemogućiti *DELETE* operaciju za tablicu *BaggageHistory* kako bi se spriječilo brisanje redaka, te onemogućiti *UPDATE* operaciju u svim slučajevima, osim kada se radi o zatvaranju gornje granice atributa *validity_period* čija je vrijednost *NULL*.

U Programskom kodu 4.5 možemo vidjeti implementaciju okidač funkcije *bh_no_delete()* i okidača *trg_bh_no_delete*. Ova funkcija i pripadajući okidač se aktiviraju prije svake *DELETE* operacije na tablici *BaggageHistory* i odmah je prekidaju uz poruku o grešci.

```

1 CREATE OR REPLACE FUNCTION bh_no_delete() RETURNS TRIGGER AS $$
2 BEGIN
3     RAISE EXCEPTION 'Deleting history rows is not allowed.';
4 END;
5 $$ LANGUAGE plpgsql;
6
7
8 CREATE TRIGGER trg_bh_no_delete BEFORE DELETE
9 ON BaggageHistory
10 FOR EACH ROW
11 EXECUTE FUNCTION bh_no_delete();

```

Programski kod 4.5: PL/pgSQL¹ i SQL kod za implementaciju okidač funkcije *bh_no_delete()* i okidača *trg_bh_no_delete*.

U Programskom kodu 4.6 prikazana je implementacija okidač funkcije *bh_no_update()* i okidača *trg_bh_no_update* koje su odgovorne za kontrolu *UPDATE* operacije nad tablicom *BaggageHistory*. Ovaj okidač se aktivira prije svake *UPDATE* operacije i prvo provjerava je li redak koji se pokušava ažurirati već povijesni (tj. gornja granica od *validity_period* nije *NULL*), u tom slučaju okidač će zabraniti promjenu. Ako je redak trenutno aktivan (tj. gornja granica od *validity_period* je *NULL*), okidač će dozvoliti promjenu, ali isključivo na način da se zatvori njegov vremenski interval. Bilo kakav pokušaj promjene drugih podataka (npr. statusa ili lokacije) na postojećem retku bit će zabranjen.

```

1 CREATE OR REPLACE FUNCTION bh_no_update() RETURNS TRIGGER AS $$
2 BEGIN
3     IF upper(OLD.validity_period) IS NOT NULL THEN
4         RAISE EXCEPTION 'Historical rows are immutable.';
5     END IF;
6
7     IF NEW.baggage_id IS DISTINCT FROM OLD.baggage_id OR
8        NEW.location_id IS DISTINCT FROM OLD.location_id OR
9        NEW.flight_id IS DISTINCT FROM OLD.flight_id OR

```

¹PL/pgSQL je proceduralni jezik koji dodaje programibilnu logiku SQL-u.

```

10     NEW.status IS DISTINCT FROM OLD.status OR
11     lower(NEW.validity_period) IS DISTINCT FROM lower(OLD.
validity_period) THEN
12         RAISE EXCEPTION 'Only closing the active interval is
allowed; other fields are immutable';
13     END IF;
14
15     IF upper(NEW.validity_period) IS NULL THEN
16         RAISE EXCEPTION 'Only closing the active interval is allowed';
17     END IF;
18
19     IF lower(NEW.validity_period) >= upper(NEW.validity_period)
THEN
20         RAISE EXCEPTION 'Upper bound must be greater than start time';
21     END IF;
22
23     RETURN NEW;
24 END;
25 $$ LANGUAGE plpgsql;
26
27
28 CREATE TRIGGER trg_bh_no_update BEFORE UPDATE
29 ON BaggageHistory
30 FOR EACH ROW
31 EXECUTE FUNCTION bh_no_update();

```

Programski kod 4.6: PL/pgSQL i SQL kod za implementaciju okidač funkcije *bh_no_update()* i okidača *trg_bh_no_update*.

4.2.2 Kreiranje funkcija

Funkcija *register_baggage_and_itinerary()* implementirana je kako bi se pojednostavio proces unosa nove prtljage i njenog plana puta, te joj je cilj osigurati da svaka prtljaga ima svoj plan puta. Prilikom poziva, ona izvršava dvije operacije: prvo kreira zapis o samoj prtljazi u tablici *Baggage*, a zatim na temelju proslijeđenih letova kreira njen plan puta u tablici *BaggageItinerary*. Iako ova funkcija ne implementira direktno temporalnu logiku, ona je ključan preduvjet za rad sustava. Tek nakon što su osnovni podaci kreirani, moguće je pratiti povijest prtljage pomoću temporalnih mehanizama. Implementacija funkcije *register_baggage_and_itinerary()* prikazana je u Programskom kodu 4.7.

```

1 CREATE OR REPLACE FUNCTION register_baggage_and_itinerary(
2     p_passenger_id BIGINT,
3     p_baggage_tag_id VARCHAR(10),
4     p_flight_ids BIGINT[]
5 )
6 RETURNS BIGINT AS $$
7 DECLARE
8     v_baggage_id BIGINT;
9     flight_id BIGINT;
10    segment_counter INT := 1;

```

```

11 BEGIN
12     -- Kreiraj prtljagu
13     INSERT INTO Baggage (baggage_tag_id, passenger_id)
14     VALUES (p_baggage_tag_id, p_passenger_id)
15     RETURNING baggage_id INTO v_baggage_id;
16
17     -- Kreiraj plan puta
18     IF p_flight_ids IS NOT NULL THEN
19         FOREACH flight_id IN ARRAY p_flight_ids
20         LOOP
21             INSERT INTO BaggageItinerary (baggage_id, flight_id,
segment_number)
22             VALUES (v_baggage_id, flight_id, segment_counter);
23             segment_counter := segment_counter + 1;
24         END LOOP;
25     END IF;
26
27     RETURN v_baggage_id;
28 END;
29 $$ LANGUAGE plpgsql;

```

Programski kod 4.7: PL/pgSQL kod za implementaciju funkcije *register_baggage_and_itinerary()*.

Funkcija *log_baggage_status()* je centralna funkcija temporalne logike ovog modela. Svaki put kada se dogodi promjena stanja prtljage (npr. skeniranje na check-in šalteru, utovar u avion, itd.) poziva se ova funkcija. Njen zadatak nije da izmijeni (*UPDATE*) stari podatak, već da sačuva povijest slijedeći *append-only princip*². Prvo pronalazi onaj redak u tablici *BaggageHistory* za danu prtljagu koji je trenutno aktivan (*upper(Validity_Period)=NULL*). Zatim se taj, sada već prošli, vremenski interval „zatvara” tako da se njegova gornja granica postavi na vrijeme novog događaja. Na kraju, funkcija kreira potpuno novi redak u tablici, koji predstavlja novo, sada aktivno stanje. Vremenski interval za novi redak počinje vremenom novog događaja i nema definiranu gornju granicu, što signalizira da je to trenutno aktivno stanje. Implementacija ove funkcije može se vidjeti u Programskom kodu 4.8.

```

1 CREATE OR REPLACE FUNCTION log_baggage_status (
2     p_baggage_tag_id VARCHAR(10),
3     p_new_location_id BIGINT,
4     p_new_status baggage_status,
5     p_new_event_time TIMESTAMPTZ,
6     p_new_flight_id BIGINT DEFAULT NULL
7 )
8 RETURNS BIGINT AS $$
9 DECLARE
10     v_baggage_id BIGINT;
11     v_new_history_id BIGINT;
12 BEGIN

```

²*Append-only princip* je princip pohrane podataka gdje se novi podaci mogu samo dodavati u postojeći skup, a već upisani podaci se ne mogu mijenjati niti brisati.

```

13  -- Pronalazi baggage_id na temelju p_baggage_tag_id
14  SELECT baggage_id INTO v_baggage_id
15  FROM Baggage
16  WHERE baggage_tag_id = p_baggage_tag_id;
17
18  -- Ako prtljaga s tom oznakom ne postoji, prekini operaciju s
   greskom
19  IF v_baggage_id IS NULL THEN
20      RAISE EXCEPTION 'Prtljaga s oznakom % ne postoji.',
p_baggage_tag_id;
21  END IF;
22
23  -- Zatvori prethodni, trenutno aktívni vremenski interval (ako
   postoji)
24  UPDATE BaggageHistory
25  SET validity_period = tstzrange(lower(validity_period),
p_new_event_time)
26  WHERE baggage_id = v_baggage_id
27      AND upper(validity_period) IS NULL;
28
29  -- Unesi novi, sada aktívni zapis u povijest
30  INSERT INTO BaggageHistory (baggage_id, location_id, flight_id,
   status, validity_period)
31  VALUES (
32      v_baggage_id,
33      p_new_location_id,
34      p_new_flight_id,
35      p_new_status,
36      tstzrange(p_new_event_time, NULL)
37  )
38  RETURNING history_id INTO v_new_history_id;
39
40  RETURN v_new_history_id;
41  END;
42  $$ LANGUAGE plpgsql;

```

Programski kod 4.8: PL/pgSQL kod za implementaciju funkcije `log_baggage_status()`.

4.3 Upiti

Kako bi se demonstrirala snaga i praktična primjena implementiranog modela, kreirano je nekoliko realističnih scenarija koji simuliraju stvarne situacije u zračnom prometu. Neki od scenarija su: uspješan direktan let, putovanje s presjedanjem, prtljaga greškom ukrcana na krivi let i prtljaga koja nije stigla na odredište. Navedeni scenariji i njihova implementacija neće biti detaljnije objašnjeni u ovom radu, ali njihov potpuni implementacijski kod dostupan je na ranije priloženoj GitHub poveznici.

Na temelju podataka unesenih kroz ove scenarije, moguće je postaviti kompleksne upite na koje standardne baze ne bi mogle jednostavno odgovoriti. U nastavku su

prikazani i objašnjeni neki od mnoštva mogućih upita.

Upit o trenutnom stanju

Upit o trenutnom stanju je najosnovniji upit koji svaki sustav za praćenje mora podržavati i ovom slučaju nam omogućuje da saznamo gdje se trenutno nalazi prtljaga X i koji je njen status. U temporalnom modelu, trenutno stanje je onaj povijesni zapis čiji vremenski interval *validity_period* još uvijek nije zatvoren i ima vrijednost *NULL*.

U Programskom kodu 4.9 prikazana je implementacija upita o trenutnom stanju. Tablica *BaggageHistory* povezana je s ostalim tablicama koristeći *JOIN* klauzulu kako bi se dohvatili čitljivi podaci umjesto ID-jeva. Za povezivanje s tablicom *Airport* korištena je *LEFT JOIN* klauzula koja osigurava da upit vrati rezultat i u slučaju kada se radi o lokacijama koje nisu vezane uz zračnu luku (npr. zrakoplov). Upravo zbog toga koristi se i funkcija *COALESCE* koja u slučaju nepostojanja zračne luke ispisuje „N/A” umjesto prazne vrijednosti. Ključan dio ovog upita je *WHERE* klauzula koja filtrira i vraća samo onaj redak koji predstavlja trenutno važeće stanje za odabranu prtljagu. Rezultat ovog upita prikazan je na Slici 4.2.

```

1 SELECT
2   b.baggage_tag_id AS "Oznaka prtljage",
3   bh.status AS "Trenutni status",
4   l.location_description AS "Trenutna lokacija",
5   COALESCE(a.iata_code, 'N/A') AS "Aerodrom",
6   to_char(lower(bh.validity_period), 'YYYY-MM-DD HH24:MI') AS "
   Vrijeme statusa"
7 FROM
8   BaggageHistory bh
9 JOIN Baggage b ON bh.baggage_id = b.baggage_id
10 JOIN Location l ON bh.location_id = l.location_id
11 LEFT JOIN Airport a ON l.airport_id = a.airport_id
12 WHERE
13   upper(bh.validity_period) IS NULL -- Samo trenutno vazeci status
14   AND b.baggage_tag_id = 'OU77889900'; -- Ovdje unesite oznaku
   prtljage

```

Programski kod 4.9: SQL kod za upit o trenutnom stanju prtljage.

	Oznaka prtljage character varying (10)	Trenutni status baggage_status	Trenutna lokacija text	Aerodrom character varying	Vrijeme statusa text
1	OU77889900	CLAIMED	Baggage Claim Carousel 12	JFK	2025-08-26 02:25

Slika 4.2: Rezultat upita o trenutnom stanju prtljage *OU77889900*.

Point-in-Time upit

Cilj Point-in-Time upita je omogućiti uvid u lokaciju i stanje prtljage u bilo kojem zadanom trenutku u prošlosti. Iako je implementacija ovog upita (Programski

kod 4.10) vrlo slična implementaciji za upit o trenutnom stanju (Programski kod 4.9), bitno se razlikuje u uvjetu unutar *WHERE* klauzule. Umjesto provjere trenutno važećeg stanja, ovaj uvjet koristi operator sadržavanja (*@>*) koji pronalazi točno onaj zapis čiji je vremenski interval bio aktivan u zadanom trenutku. Rezultat je precizan prikaz stanja prtljage u trenutku u prošlosti kao što je vidljivo na Slici 4.3.

```

1 SELECT
2   b.baggage_tag_id AS "Oznaka prtljage",
3   to_char('2025-08-25 16:00:00'::TIMESTAMP, 'YYYY-MM-DD HH24:MI:SS') AS "Trazeno vrijeme",
4   bh.status AS "Status u trazeno vrijeme",
5   l.location_description AS "Lokacija",
6   COALESCE(a.iata_code, '') AS "Aerodrom",
7   COALESCE(f.flight_number, '') AS "Let"
8 FROM BaggageHistory bh
9 JOIN Baggage b ON bh.baggage_id = b.baggage_id
10 JOIN Location l ON bh.location_id = l.location_id
11 LEFT JOIN Airport a ON l.airport_id = a.airport_id
12 LEFT JOIN Flight f ON bh.flight_id = f.flight_id
13 WHERE
14   b.baggage_tag_id = 'OU77889900' -- Ovdje unesite zeljenu oznaku prtljage
15   AND bh.validity_period @> '2025-08-25 16:00:00'::TIMESTAMP; -- Ovdje unesite zeljeni trenutak

```

Programski kod 4.10: SQL kod za upit o stanju prtljage u određenom trenutku.

	Oznaka prtljage character varying (10) 🔒	Traženo vrijeme text 🔒	Status u traženo vrijeme baggage_status 🔒	Lokacija text 🔒	Aerodrom character varying 🔒	Let character varying 🔒
1	OU77889900	2025-08-25 16:00:00	IN_SORTING_FACILITY	Arrival Sorting Facility	FRA	

Slika 4.3: Rezultat upita o stanju prtljage *OU77889900* u trenutku *2025-08-25 16:00:00*.

Rekonstrukcija povijesti

Za razliku od prethodnih upita koji su izdvajali samo jedan specifičan trenutak, upit o rekonstrukciji povijesti prtljage prikazuje cjelokupan slijed događaja za odabranu prtljagu. Njegova implementacija, prikazana u Programskom kodu 4.11, koristi jednostavan uvjet u *WHERE* klauzuli koji filtrira zapise isključivo prema oznaci prtljage. Nakon toga, koristi se *ORDER BY* klauzula, koja sortira zapise prema vremenu njihovog nastanka i osigurava točan kronološki prikaz događaja. Za povezivanje tablica korišteni su slični koncepti kao i u prethodnim upitima. Rezultat ovog upita je cjelovit prikaz kretanja prtljage, kao što je prikazano na Slici 4.4.

```

1 SELECT
2     to_char(lower(bh.validity_period), 'YYYY-MM-DD HH24:MI') AS "
    Vrijeme",
3     bh.status AS "Status",
4     l.location_description AS "Lokacija",
5     COALESCE(a.iata_code || ' (' || a.airport_name || ')', '') AS "
    Aerodrom",
6     COALESCE(f.flight_number || ' (' || origin_a.iata_code || ' ->
    ' || dest_a.iata_code || ')', '') AS "Let"
7 FROM BaggageHistory bh
8 JOIN Baggage b ON bh.baggage_id = b.baggage_id
9 JOIN Location l ON bh.location_id = l.location_id
10 LEFT JOIN Airport a ON l.airport_id = a.airport_id
11 LEFT JOIN Flight f ON bh.flight_id = f.flight_id
12 LEFT JOIN Airport origin_a ON f.origin_airport_id = origin_a.
    airport_id
13 LEFT JOIN Airport dest_a ON f.destination_airport_id = dest_a.
    airport_id
14 WHERE b.baggage_tag_id = 'OU77889900' -- Ovdje unesite oznaku
    prtljage
15 ORDER BY lower(bh.validity_period) ASC;

```

Programski kod 4.11: SQL kod za upit o rekonstrukciji puta prtljage.

	Vrijeme text	Status baggage_status	Lokacija text	Aerodrom text	Let text
1	2025-08-25 09:40	CHECKED_IN	Check-in Counter 12	ZAG (Franjo Tuđman Airport)	
2	2025-08-25 09:55	IN_SORTING_FACILITY	Departure Sorting Facility	ZAG (Franjo Tuđman Airport)	
3	2025-08-25 10:50	LOADED	Loading Area Gate A10	ZAG (Franjo Tuđman Airport)	OU416 (ZAG -> FRA)
4	2025-08-25 11:10	IN_TRANSIT	Onboard Aircraft		OU416 (ZAG -> FRA)
5	2025-08-25 12:45	UNLOADED	Unloading Area Ramp C15	FRA (Frankfurt Airport)	OU416 (ZAG -> FRA)
6	2025-08-25 13:00	IN_SORTING_FACILITY	Arrival Sorting Facility	FRA (Frankfurt Airport)	
7	2025-08-25 16:30	LOADED	Loading Area Gate B22	FRA (Frankfurt Airport)	LH404 (FRA -> JFK)
8	2025-08-25 17:00	IN_TRANSIT	Onboard Aircraft		LH404 (FRA -> JFK)
9	2025-08-26 01:50	UNLOADED	Unloading Area Ramp E03	JFK (John F. Kennedy International)	LH404 (FRA -> JFK)
10	2025-08-26 02:15	READY_FOR_PICKUP	Baggage Claim Carousel 12	JFK (John F. Kennedy International)	
11	2025-08-26 02:25	CLAIMED	Baggage Claim Carousel 12	JFK (John F. Kennedy International)	

Slika 4.4: Rezultat upita o rekonstrukciji puta prtljage OU77889900.

Usporedba planiranog i stvarno ukrcanog broja prtljage

Ovaj kompleksni analitički upit uspoređuje planirani broj prtljage sa stvarno ukrcanim brojem za sve letove s polaskom iz određene zračne luke na određeni dan. Osim usporedbe, upit dodatno identificira koliko komada prtljage, koja je bila planirana za te letove, trenutno ima problematičan status (*DELAYED* ili *LOST*).

Zbog svoje kompleksnosti, kao što je vidljivo u Programskom kodu 4.12, upit je razbijen na manje korake uz pomoć *WITH* klauzule. Privremena tablica *Flight-sToday* definira sve relevantne letove (letovi iz određene zračne luke na određeni

dan), *PlannedCount* u tablici *BaggageItinerary* broji koliko je komada prtljage bilo planirano za svaki od tih letova, *ActualLoadedCount* pretražuje tablicu *BaggageHistory* kako bi se prebrojalo koliko je prtljage stvarno ukrcano na svaki od tih letova, *ProblemCount* kombinira plan iz *BaggageItinerary* i trenutno stanje iz *BaggageHistory*, kako bi se prebrojale sve prtljage s trenutnim statusom *DELAYED* ili *LOST*. Završni *SELECT* upit spaja sve gore navedene privremene tablice, čime se dobiva konačan i sažet izvještaj koji je vidljiv na Slici 4.5.

```

1 WITH FlightsToday AS (
2     -- Prvo definiramo opseg letova koji nas zanimaju
3     SELECT flight_id, flight_number, departure_time,
4           destination_airport_id
5     FROM Flight
6     WHERE origin_airport_id = (SELECT airport_id FROM Airport WHERE
7           iata_code = 'ZAG')
8           AND date(departure_time) = '2025-08-25'
9 ),
10 PlannedCount AS (
11     -- Prebroji planiranu prtljagu po letu
12     SELECT flight_id, count(baggage_id) AS planned_bags
13     FROM BaggageItinerary
14     WHERE flight_id IN (SELECT flight_id FROM FlightsToday)
15     GROUP BY flight_id
16 ),
17 ActualLoadedCount AS (
18     -- Prebroji stvarno ukrcanu prtljagu po letu
19     SELECT flight_id, count(DISTINCT baggage_id) AS actual_bags
20     FROM BaggageHistory
21     WHERE flight_id IN (SELECT flight_id FROM FlightsToday) AND
22           status = 'LOADED'
23     GROUP BY flight_id
24 ),
25 ProblemCount AS (
26     -- Prebroji prtljagu koja je trebala biti na letu, ali je
27     -- završila kao DELAYED/LOST
28     SELECT
29         bi.flight_id,
30         count(*) AS problem_bags
31     FROM BaggageItinerary bi
32     -- Nadi trenutni status za svaku prtljagu
33     JOIN (
34         SELECT DISTINCT ON (baggage_id) baggage_id, status
35         FROM BaggageHistory
36         ORDER BY baggage_id, lower(validity_period) DESC
37     ) AS CurrentStatuses ON bi.baggage_id = CurrentStatuses.
38     baggage_id
39     WHERE bi.flight_id IN (SELECT flight_id FROM FlightsToday)
40           AND CurrentStatuses.status IN ('DELAYED', 'LOST')
41     GROUP BY bi.flight_id
42 )
43 SELECT
44     ft.flight_number AS "Let",
45     to_char(ft.departure_time, 'HH24:MI') AS "Vrijeme polaska",
46     dest_a.iata_code || ' (' || dest_a.airport_name || ') ' AS "

```

```

    Odredi te",
42     COALESCE(pc.planned_bags, 0) AS "Planirano",
43     COALESCE(alc.actual_bags, 0) AS "Ukrcano",
44     COALESCE(alc.actual_bags, 0) - COALESCE(pc.planned_bags, 0) AS
    "Razlika",
45     COALESCE(prob.problem_bags, 0) AS "DELAYED/LOST"
46 FROM FlightsToday ft
47 LEFT JOIN Airport dest_a ON ft.destination_airport_id = dest_a.
    airport_id
48 LEFT JOIN PlannedCount pc ON ft.flight_id = pc.flight_id
49 LEFT JOIN ActualLoadedCount alc ON ft.flight_id = alc.flight_id
50 LEFT JOIN ProblemCount prob ON ft.flight_id = prob.flight_id
51 ORDER BY ft.departure_time;

```

Programski kod 4.12: SQL kod za upit koji uspoređuje koliko je prtljage trebalo biti ukrcano na let, a koliko je stvarno ukrcano.

	Let character varying (6)	Vrijeme polaska text	Odredište text	Planirano bigint	Ukrcano bigint	Razlika bigint	DELAYED/LOST bigint
1	OU416	11:10	FRA (Frankfurt Airport)	4	4	0	1
2	AF1561	18:00	CDG (Paris Charles de Gaulle)	2	1	-1	1
3	BA849	18:10	LHR (London Heathrow)	0	1	1	0

Slika 4.5: Rezultat upita koji uspoređuje koliko je prtljage trebalo biti ukrcano na let, a koliko je stvarno ukrcano.

Identifikacija neukrcane prtljage

Upit za identifikaciju neukrcane prtljage, čija je implementacija prikazana u Programskom kodu 4.13, pronalazi svu prtljagu koja je prema planu iz tablice *BaggageItinerary* trebala biti ukrcana na određeni let, a na kraju prema povijesnim zapisima iz tablice *BaggageHistory* nije bila ukrcana.

Kao i u prethodnom primjeru, korištena je *WITH* klauzula kako bi se upit razbio u manje korake. Prvi korak, *TargetFlight* odabire ciljani let, *PlannedBaggage* pronalazi svu prtljagu planiranu za taj let, a *ActuallyLoadedBaggage* pronalazi svu prtljagu koja je stvarno ukrcana na taj let. Na kraju, završni *SELECT* upit, koristeći *WHERE ... NOT IN* klauzulu, uspoređuje ta dva skupa i vraća samo onu prtljagu koja se nalazi u *PlannedBaggage*, ali ne i u *ActuallyLoadedBaggage*. Rezultat ovog upita prikazan je na Slici 4.6.

```

1 WITH
2 TargetFlight AS (
3     -- Odaberi ciljani let po broju i datumu polaska
4     SELECT flight_id
5     FROM Flight
6     WHERE flight_number = 'AF1561' AND date(departure_time) = '
        2025-08-25'
7 ),

```

```

8 PlannedBaggage AS (
9   -- Pronadi svu prtljagu koja je planirana za taj let
10  SELECT bi.baggage_id
11  FROM BaggageItinerary bi
12  JOIN TargetFlight tf ON bi.flight_id = tf.flight_id
13 ),
14 ActuallyLoadedBaggage AS (
15   -- Pronadi svu prtljagu koja je stvarno ukrcana (status 'LOADED')
16   na taj let
17  SELECT bh.baggage_id
18  FROM BaggageHistory bh
19  JOIN TargetFlight tf ON bh.flight_id = tf.flight_id
20  WHERE bh.status = 'LOADED'
21 )
22 SELECT
23   b.baggage_tag_id AS "Oznaka prtljage",
24   p.first_name || ' ' || p.last_name AS "Putnik"
25 FROM PlannedBaggage pb
26 JOIN Baggage b ON pb.baggage_id = b.baggage_id
27 JOIN Passenger p ON b.passenger_id = p.passenger_id
28 WHERE
29   pb.baggage_id NOT IN (SELECT baggage_id FROM
30     ActuallyLoadedBaggage);

```

Programski kod 4.13: SQL kod za identifikaciju neukrcane prtljage.

	Oznaka prtljage character varying (10)	Putnik text
1	AF11122233	Ivana Petrović

Slika 4.6: Rezultat upita za identifikaciju neukrcane prtljage.

Usporedba povijesti i plana puta prtljage

Svrha ovog upita je za odabrani komad prtljage vizualno, redak po redak, usporediti stvarni let zabilježen u povijesti s onim koji je za taj segment puta bio originalno planiran. Ovim upitom rješavamo problem kako usporediti planirane segmente putovanja sa stvarnim, kronološkim nizom događaja.

Kao što je prikazano u Programskom kodu 4.14 upit započinje *WITH* klauzulom kako bi se problem podijelio u manje korake. *EnrichedHistory* svakom letu zabilježenom u tablici *BaggageHistory*, pomoću funkcije *DENSE_RANK()*, dodjeljuje redni broj segmenta. Funkcija to radi tako da za svaku prtljagu zasebno (*PARTITION BY baggage_id*) sortira letove prema vremenu polaska i dodjeljuje im brojeve od 1 do n. Nakon toga, završni *SELECT* spaja *EnrichedHistory* s planom puta *BaggageItinerary* upravo preko tih rednih brojeva segmenata. Time se za svaki korak putovanja u istom retku dobiva i planirani i stvarni let, što omogućuje direktnu usporedbu, kao na Slici 4.7.

```

1 WITH
2 EnrichedHistory AS (
3     SELECT
4         bh.baggage_id,
5         bh.flight_id AS actual_flight_id,
6         CASE
7             WHEN bh.flight_id IS NOT NULL THEN DENSE_RANK() OVER (
PARTITION BY bh.baggage_id ORDER BY f.departure_time)
8             ELSE NULL
9         END AS actual_segment_number,
10        to_char(lower(bh.validity_period), 'YYYY-MM-DD HH24:MI') AS
event_time,
11        bh.status,
12        l.location_description,
13        a.iata_code AS airport_code,
14        f.flight_number || ' (' || fo.iata_code || ' -> ' || fd.
iata_code || ') ' AS actual_flight_info
15 FROM BaggageHistory bh
16 JOIN Baggage b ON b.baggage_id = bh.baggage_id
17 JOIN Location l ON l.location_id = bh.location_id
18 LEFT JOIN Airport a ON a.airport_id = l.airport_id
19 LEFT JOIN Flight f ON f.flight_id = bh.flight_id
20 LEFT JOIN Airport fo ON fo.airport_id = f.origin_airport_id
21 LEFT JOIN Airport fd ON fd.airport_id = f.
destination_airport_id
22 WHERE b.baggage_tag_id = 'OU44556677' -- Unesite oznaku
prtljage
23 )
24 SELECT
25     eh.event_time AS "Vrijeme",
26     eh.status AS "Status",
27     eh.location_description AS "Lokacija",
28     COALESCE(eh.airport_code, '') AS "Aerodrom",
29     COALESCE(eh.actual_flight_info, '') AS "Stvarni Let",
30     COALESCE(p_flight.flight_number || ' (' || p_origin.iata_code
|| ' -> ' || p_dest.iata_code || ')', '') AS "Planirani Let"
31 FROM EnrichedHistory eh
32 LEFT JOIN BaggageItinerary bi ON eh.baggage_id = bi.baggage_id AND
eh.actual_segment_number = bi.segment_number
33 LEFT JOIN Flight p_flight ON bi.flight_id = p_flight.flight_id
34 LEFT JOIN Airport p_origin ON p_flight.origin_airport_id = p_origin
.airport_id
35 LEFT JOIN Airport p_dest ON p_flight.destination_airport_id =
p_dest.airport_id
36 ORDER BY eh.event_time ASC;

```

Programski kod 4.14: SQL kod za usporedbu povijesti i plana puta prtljage.

	Vrijeme text	Status baggage_status	Lokacija text	Aerodrom character var	Stvarni Let text	Planirani Let text
1	2025-08-25 09:50	CHECKED_IN	Check-in Counter 12	ZAG		
2	2025-08-25 10:05	IN_SORTING_FACILITY	Departure Sorting Facility	ZAG		
3	2025-08-25 10:52	LOADED	Loading Area Gate A10	ZAG	OU416 (ZAG -> FRA)	OU416 (ZAG -> FRA)
4	2025-08-25 11:10	IN_TRANSIT	Onboard Aircraft		OU416 (ZAG -> FRA)	OU416 (ZAG -> FRA)
5	2025-08-25 12:45	UNLOADED	Unloading Area Ramp C15	FRA	OU416 (ZAG -> FRA)	OU416 (ZAG -> FRA)
6	2025-08-25 13:05	IN_SORTING_FACILITY	Arrival Sorting Facility	FRA		
7	2025-08-25 16:40	LOADED	Loading Area Gate B22	FRA	BA257 (FRA -> LHR)	LH404 (FRA -> JFK)
8	2025-08-25 17:10	IN_TRANSIT	Onboard Aircraft		BA257 (FRA -> LHR)	LH404 (FRA -> JFK)
9	2025-08-25 18:15	UNLOADED	Unloading Area Ramp E03	LHR	BA257 (FRA -> LHR)	LH404 (FRA -> JFK)
10	2025-08-25 18:25	IN_SORTING_FACILITY	Arrival Sorting Facility	LHR		
11	2025-08-25 18:50	DELAYED	Lost and Found Office	LHR		

Slika 4.7: Rezultat upita usporedba povijesti i plana puta prtljage OU44556677.

Analiza efikasnosti procesa ukrcaja prtljage

Jedan od ključnih pokazatelja efikasnosti zračne luke je pravovremenost ukrcaja prtljage. Upit prikazan u Programskom kodu 4.15 analizira upravo taj pokazatelj, uspoređujući planirano vrijeme polaska sa stvarnim vremenom završetka ukrcaja.

I u ovom upitu korištena je *WITH* klauzula kako bi se rješenje podijelilo u nekoliko koraka. U prvom koraku, *LastBagLoaded*, pretražuje tablicu *BaggageHistory* i za svaki *flight_id* pronalazi vrijeme kada je zadnji komad prtljage dobio status *LOADED*, što predstavlja trenutak završetka ukrcaja. Završni *SELECT* upit spaja te podatke s tablicom letova te pomoću *CASE* naredbe izračunava razliku, dajući jasan status o pravovremenosti ukrcaja za svaki let, kao što je vidljivo na Slici 4.8.

```

1 WITH
2 LastBagLoaded AS (
3     -- Za svaki let, pronadi vrijeme ukrcaja ukrcaja zadnje
   prtljage
4     SELECT
5         flight_id,
6         MAX(lower(validity_period)) AS last_bag_load_time
7     FROM BaggageHistory
8     WHERE status = 'LOADED' AND flight_id IS NOT NULL
9     GROUP BY flight_id
10 )
11 -- Usporedi to vrijeme s planiranim vremenom polaska za sve letove
   s odabranog aerodroma na dani datum
12 SELECT
13     f.flight_number AS "Let",
14     dest.iata_code AS "Odredi te",
15     to_char(f.departure_time, 'HH24:MI:SS') AS "Planirani polazak",
16     COALESCE(to_char(lbl.last_bag_load_time, 'HH24:MI:SS'), 'Nema
   ukrcaje prtljage') AS "Ukrcaj zadnje prtljage",
17     -- Izracunaj razliku
18     CASE
19         WHEN lbl.last_bag_load_time IS NULL THEN 'N/A'

```

```

20         WHEN f.departure_time >= lbl.last_bag_load_time THEN
21             to_char(f.departure_time - lbl.last_bag_load_time, 'MI"
m "SS"s prije polaska"')
22         ELSE
23             to_char(lbl.last_bag_load_time - f.departure_time, 'MI"
m "SS"s nakon polaska (kasnjenje)"')
24     END AS "Status ukrcaja"
25 FROM Flight f
26 LEFT JOIN LastBagLoaded lbl ON f.flight_id = lbl.flight_id
27 JOIN Airport dest ON f.destination_airport_id = dest.airport_id
28 WHERE
29     f.origin_airport_id = (SELECT airport_id FROM Airport WHERE
iata_code = 'ZAG') -- Filtriraj po aerodromu
30     AND date(f.departure_time) = '2025-08-25' -- Filtriraj po
datumu
31 ORDER BY
32     f.departure_time;

```

Programski kod 4.15: SQL kod za analizu efikasnosti procesa ukrcaja prtljage.

	Let character varying (6)	Odredište character varying (3)	Planirani polazak text	Ukrcaj zadnje prtljage text	Status ukrcaja text
1	OU416	FRA	11:10:00	10:52:00	18m 00s prije polaska
2	AF1561	CDG	18:00:00	18:05:00	05m 00s nakon polaska (kasnjenje)
3	BA849	LHR	18:10:00	17:45:00	25m 00s prije polaska

Slika 4.8: Rezultat upita o analizi efikasnosti procesa ukrcaja prtljage.

Analiza prosječnog čekanja prtljage

Ovaj analitički upit mjeri efikasnost procesa isporuke prtljage za svaku dolaznu zračnu luku iz perspektive putnika, izračunavajući prosječno vrijeme koje prođe od slijetanja aviona do trenutka kada je prtljaga spremna za preuzimanje.

Implementacija upita, prikazana u Programskom kodu 4.16, započinje *WITH* klauzulom, koja razbija problem u nekoliko koraka. *ReadyForPickupEvents* pronalazi točan trenutak kada je svaka prtljaga dobila status *READY_FOR_PICKUP*, dok *LastFlightInfo* saznaje vrijeme slijetanja zadnjeg leta na kojem je prtljaga bila. Završni *SELECT* upit spaja ta dva podatka, izračunava vremensku razliku za svaku pojedinu prtljagu i na kraju koristeći *GROUP BY* klauzulu i funkciju *AVG* računa prosjek vremenskih razlika po zračnoj luci. Time se dobiva jasan pokazatelj kvalitete usluge, kao što je prikazano na Slici 4.9.

```

1 WITH
2 ReadyForPickupEvents AS (
3     -- Pronadi sve događaje kada je prtljaga bila spremna za
preuzimanje, zajedno s aerodromom
4     SELECT
5         bh.baggage_id,
6         lower(bh.validity_period) AS ready_time,

```

```

7         l.airport_id
8     FROM BaggageHistory bh
9     JOIN Location l ON bh.location_id = l.location_id
10    WHERE bh.status = 'READY_FOR_PICKUP'
11 ),
12 LastFlightInfo AS (
13     -- Za svaku prtljagu, pronadi ID zadnjeg leta na kojem je bila
14     (zadnji UNLOADED status)
15     SELECT DISTINCT ON (baggage_id)
16         baggage_id,
17         flight_id
18     FROM BaggageHistory
19     WHERE status = 'UNLOADED'
20     ORDER BY baggage_id, lower(validity_period) DESC
21 )
22 -- Spoji podatke o vremenu preuzimanja s podacima o letu i
23 -- izracunaj prosjek po dolaznom aerodromu
24 SELECT
25     a.iata_code || ' (' || a.airport_name || ')' AS "Aerodrom
26     dolaska",
27     to_char(AVG(rfp.ready_time - f.arrival_time), 'MI" minuta i "SS
28     " sekundi") AS "Prosjek cekanja prtljage"
29 FROM ReadyForPickupEvents rfp
30 JOIN LastFlightInfo lfi ON rfp.baggage_id = lfi.baggage_id
31 JOIN Flight f ON lfi.flight_id = f.flight_id
32 JOIN Airport a ON rfp.airport_id = a.airport_id
33 GROUP BY a.airport_id, a.iata_code, a.airport_name
34 ORDER BY AVG(rfp.ready_time - f.arrival_time) DESC;

```

Programski kod 4.16: SQL kod za upit o prosječnom čekanju prtljage.

	Aerodrom dolaska text	Prosjek čekanja prtljage text
1	CDG (Paris Charles de Gaulle)	35 minuta i 00 sekundi
2	JFK (John F. Kennedy International)	30 minuta i 00 sekundi
3	FRA (Frankfurt Airport)	26 minuta i 30 sekundi

Slika 4.9: Rezultat upita o prosječnom čekanju prtljage.

Literatura

- [1] T. CONNOLLY, C. BEGG, *Database Systems: A Practical Approach to Design, Implementation, and Management*, Pearson Education, Harlow, 2015.
- [2] R. ELMASRI, S. B. NAVATHE, *Fundamentals of Database Systems, seventh edition*, Pearson Education, 2016.
- [3] H. GARCIA-MOLINA, J. D. ULLMAN, J. WIDOM, *Database Systems: The Complete Book*, Prentice Hall, New Jersey, 2009.
- [4] R. T. SNODGRASS, *Developing Time-Oriented Database Applications in SQL*, Morgan Kaufmann Publishers, San Francisco, 1999.
- [5] C. S. Jensen, *Temporal Database Management*, Aalborg University, Aalborg, 2000.
- [6] *What is a Database?*, dostupno na <https://www.ibm.com/think/topics/database>
- [7] *NoSQL Databases Visually Explained with Examples*, dostupno na <https://www.altexsoft.com/blog/nosql-databases/>
- [8] *Custom Temporal Tables in PostgreSQL*, dostupno na <https://medium.com/@vbilopav/custom-temporal-tables-in-postgresql-5a0e4045ab30>
- [9] *Vremenske baze podataka*, dostupno na [https://www.fer.unizg.hr/_download/repository/5._Vremenske_baze_podataka\[3\].pdf](https://www.fer.unizg.hr/_download/repository/5._Vremenske_baze_podataka[3].pdf)
- [10] *Introduction To Temporal Database*, dostupno na <https://www.geeksforgeeks.org/dbms/introduction-to-temporal-database/>
- [11] *PostgreSQL dokumentacija*, dostupno na <https://www.postgresql.org/>
- [12] *Shapes and Forms of Structured Data: SCD Types, Master Full, Master Incremental, Unitemporal, and Bitemporal*, dostupno na <https://getdozer.io/blog/data-shapes/>

Sažetak

Tema ovog diplomskog rada je primjena temporalne baze podataka u sustavu za praćenje prtljage u zračnom prometu. U ovom radu opisane su baze podataka općenito te njihovi tipovi s fokusom na relacijske baze podataka. Definirane su temporalne baze podataka i njihovi osnovni koncepti, također su objašnjene vremenske dimenzije, modeliranje temporalnih podataka u relacijskom modelu i vrste temporalnih upita. Zadnji dio rada bavi se implementacijom temporalne baze podataka za praćenje prtljage u zračnom prometu, a na bazi podataka proveden je niz temporalnih upita. Rad potvrđuje da temporalne baze podataka posjeduju rješenje za sustave u kojima je praćenje i analiza povijesnih događaja od presudne važnosti.

Ključne riječi

temporalna baza podataka, baza podataka, povijest podataka, vremenski period, praćenje prtljage, PostgreSQL

The application of temporal databases in an air traffic baggage tracking system

Summary

The topic of this master's thesis is the application of temporal databases in an air traffic baggage tracking system. This paper describes databases in general and their types with a focus on relational databases. Temporal databases and their basic concepts are also defined, along with an explanation of temporal dimensions, modeling temporal data in a relational model, and types of temporal queries. The final part of the paper deals with the implementation of a temporal database for baggage tracking in air traffic. A series of temporal queries were performed on the database. The paper confirms that temporal databases provide a key solution for systems where the tracking and analysis of historical events are of crucial importance.

Keywords

temporal database, database, data history, time period, baggage tracking, PostgreSQL

Životopis

Rođena sam 11. studenog 1999. godine u Sisku. Pohađala sam Osnovnu školu Višnjevac u Višnjevcu i kasnije završila III. Gimnaziju Osijek u Osijeku. Godine 2018. upisujem se na preddiplomski studij matematike i računarstva na Odjelu za matematiku u Osijeku. Zimski semestar 2021./2022. godine provela sam na Erasmusu+ studentskoj razmjeni u Krakovu, a u lipnju 2022. godine završavam preddiplomski studij s temom završnog rada "Van Emde Boasova stabla". Godine 2022. upisala sam diplomski studij matematike, smjer matematika i računarstvo na Fakultetu primijenjene matematike i informatike.